

Submission Plan

Pre-timebox specification gate

Before starting the four-hour implementation timer, confirm that the product requirements, acceptance criteria, technical design, decisions, API contracts, ordered tasks, and verification matrix agree. This is the compact spec-driven baseline. Do not continue expanding documentation once the implementation timebox starts unless behavior changes.

Use `main` plus one short-lived `feature/trial-booking` branch. Full GitFlow is intentionally excluded because the exercise has no separate release or hotfix lifecycle.

Four-hour implementation timebox

Time	Focus	Exit condition
0:00-0:25	Initialize, schema, Docker, migration, seed skeleton	Database starts and migrates
0:25-1:25	Booking service, constraints, atomic confirmation	Core integration tests pass
1:25-2:05	API routes and error contracts	Flow works without UI
2:05-2:40	Minimal UI and roster	End-to-end manual demo works
2:40-3:20	Edge tests and repeated race verification	Required matrix is green
3:20-3:50	README and AI_USAGE	Docs match implementation
3:50-4:00	Clean clone check and notes	Submission-ready or gaps declared

If behind schedule, cut UI polish first. Do not cut the race test, payment failure behavior, or README explanation.

README outline

1. Project summary.
2. Quick start.
3. Demo flow.
4. What was built.
5. Time spent.
6. Assumptions.
7. Architecture and data model.
8. API or server function table.
9. Booking statuses and transitions.
10. Invariants and layer responsibilities.
11. Duplicate and idempotency behavior.
12. Payment failure behavior.

13. Last-seat race approach, proof, and tradeoffs.
14. Why the take-home uses a transaction/state machine instead of a Saga.
15. Why Kafka was considered and deferred; transactional outbox as the first evolution step.
16. Tests and verification commands.
17. Deliberate scope cuts.
18. Production monitoring.
19. Next steps.

AI_USAGE outline

- AI tools used, including model/product names actually used.
- Tasks delegated to AI: requirement decomposition, schema review, test-case generation, documentation, or code assistance.
- A concrete example where AI accelerated the work.
- A concrete suggestion that was rejected or corrected, with the reasoning.
- How generated work was verified through tests, review, and runtime checks.
- What would change in a future AI workflow.

Do not write a generic endorsement of AI. Reviewers explicitly want evidence of steering, skepticism, and verification.

Walkthrough script (5-8 minutes)

0:00-0:45 - Scope and architecture

State the goal, deliberate cuts, stack, and three central invariants.

0:45-2:15 - Happy path

Show seeded data, create a booking, choose payment success, view `confirmed`, and show the roster.

2:15-3:15 - Failure and duplicate paths

Show a failed payment excluded from the roster and a duplicate attempt rejected or reused.

3:15-5:15 - Last-seat race

Show the class with three confirmed students, run the concurrent test, and show one winner plus final count four. Briefly explain the class-row lock and transaction.

5:15-6:15 - Tradeoffs

Explain why pending bookings do not reserve seats, why payment attempts are separate, and how real authorization/capture would improve the design.

6:15-7:00 - Verification and next steps

Show the test summary, monitoring ideas, actual time spent, and what more time would buy.

Final submission checklist

- Public GitHub repository; no zip file.
- README includes every requested topic.
- AI_USAGE includes every requested topic.
- Synthetic seed/setup is included.
- Test or verification instructions are copy-pasteable.
- Fresh setup has been tested.
- No secrets, local paths, build output, or real child data are committed.
- Last-seat behavior is both implemented and demonstrated.
- Saga and Kafka are described as considered production options, not falsely listed as implemented components.
- Git history is reviewable without unnecessary GitFlow branches.
- Video link is accessible without requesting permission.
- Video duration is 5-8 minutes.
- Repository default branch contains the demonstrated commit.
- Actual time spent is recorded honestly.

More-time backlog

Prioritize in this order:

1. Real authorization/capture plus void/refund reconciliation.
2. Authentication, parent ownership, and staff authorization.
3. Pending-booking expiry and cleanup.
4. Outbox-driven notifications and audit events.
5. Rate limiting and abuse protection.
6. Accessibility and UI polish.
7. Deployment, dashboards, and alerting.